

Goose AI System Map

How the three repositories fit together, from the browser request to the LLM call and back: who calls whom, which store holds what, and what runs on a schedule. `db_agent` is drawn registered and working, using the rx-goose implementation; its code is identical in rx04-goose.

● rx04-goose · FastAPI backend (live)

● rx04-goose-mcp-server · read-only MCP data service

● rx-goose · legacy predecessor

01 System overview

04 db_agent NL-to-SQL

07 MCP server

10 Endpoint inventory

02 /qna/ask lifecycle

05 RAG ingest & retrieval

08 Super-admin plane

11 rx-goose vs rx04-goose

03 Planner & agent graph

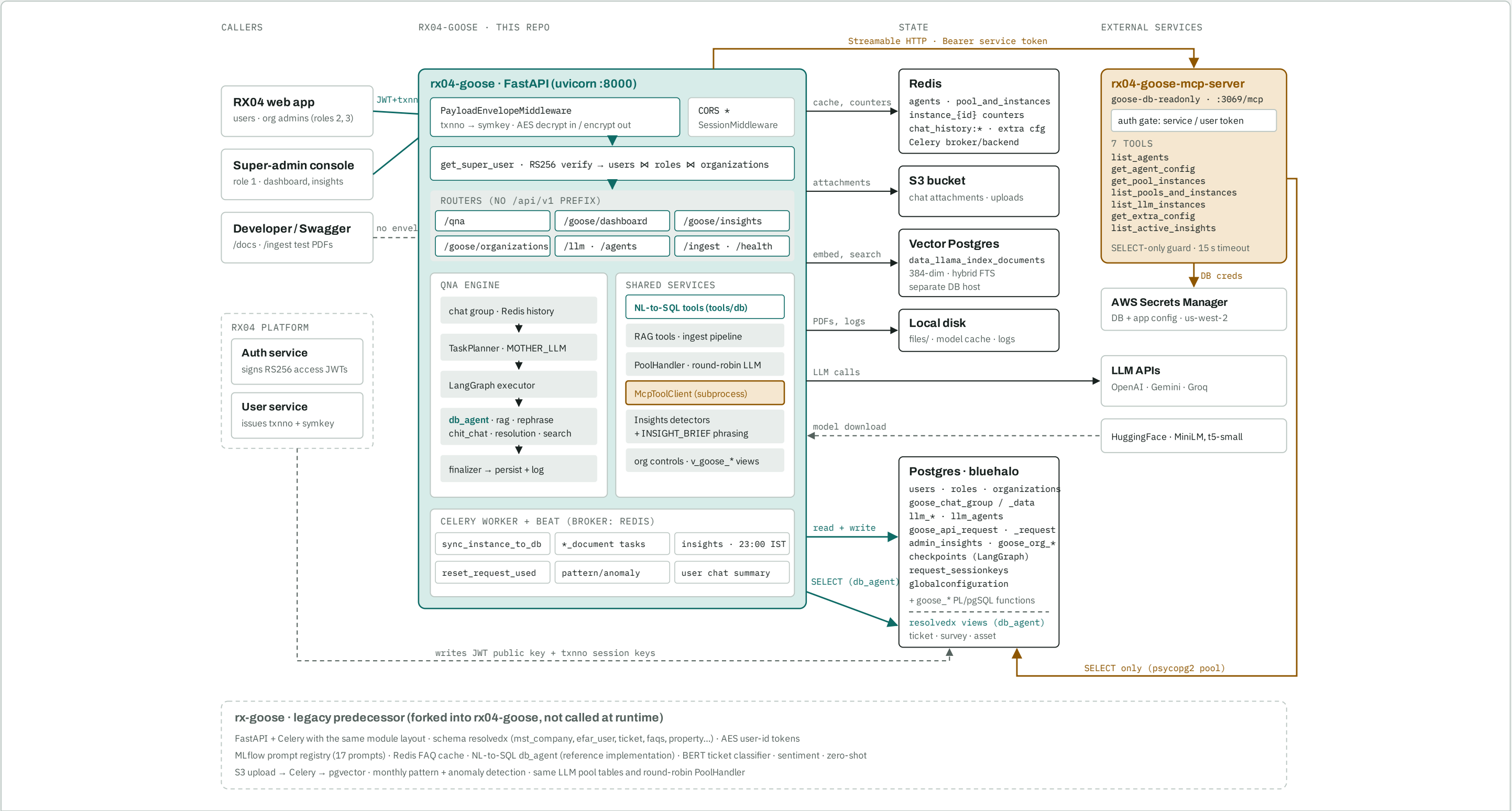
06 LLM routing

09 Background jobs

12 Gaps found in the code

Everything that talks to everything

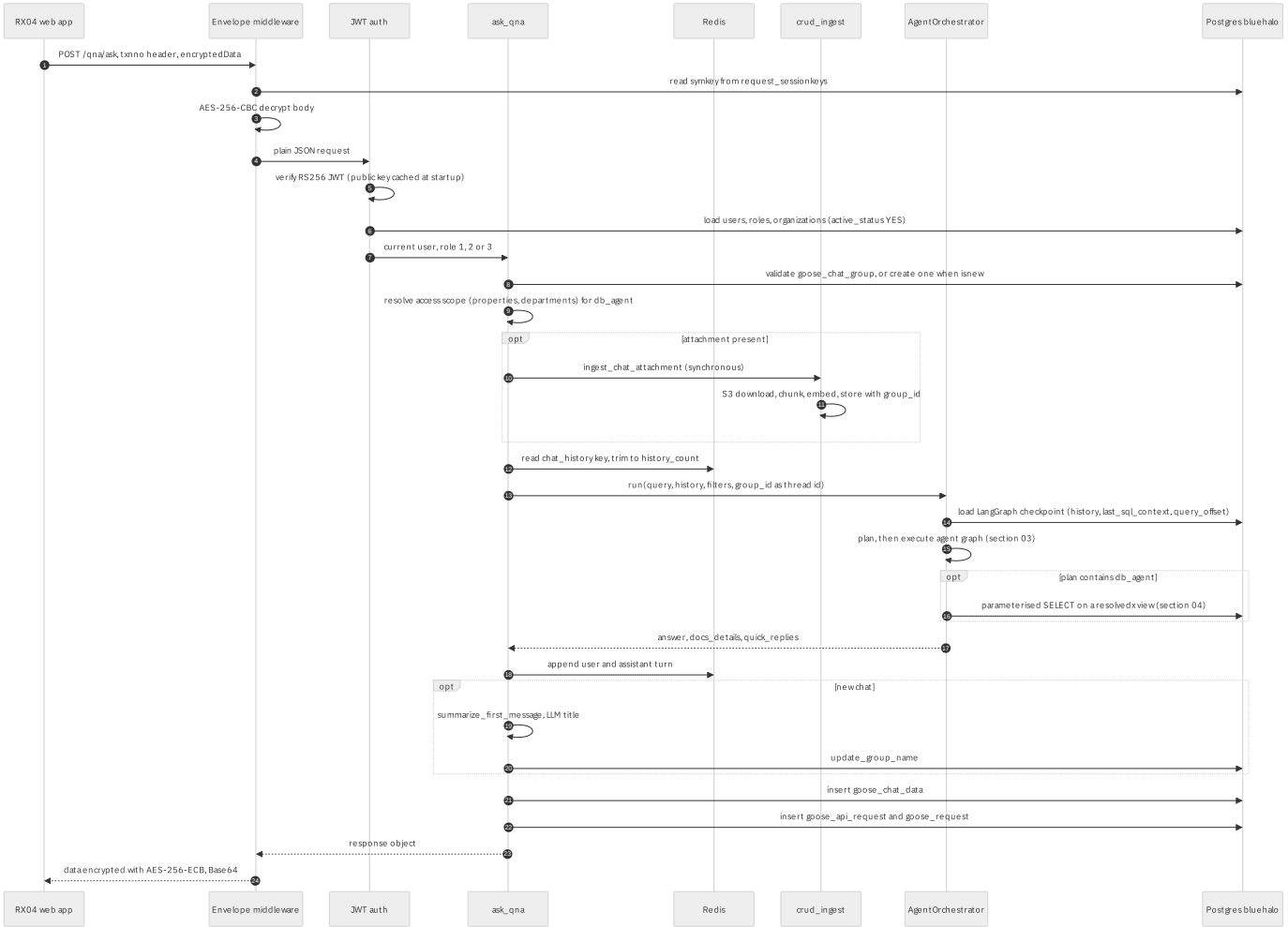
The RX04 web app calls **rx04-goose** with an RS256 JWT and an encrypted body. rx04-goose owns the AI logic and writes to Postgres directly. It reads agent config and admin insights through the **MCP server**, which can only run SELECTs. **rx-goose** is the older codebase rx04-goose was forked from; nothing calls it at runtime.



rx04-goose is the only service that holds business logic. Postgres **bluehalo** is shared by three writers and readers: RX04 platform services write the auth keys, rx04-goose reads and writes chat, logs and admin data, and the MCP server reads config and insights with SELECT only. **db_agent** runs parameterised SELECTs on the **resolvedx** ticket, survey and asset views, as it does in rx-goose.

One chat question, start to finish

The live handler is `crud/crud_qna_chitchat.py` (`CRUDQnaChitChat.ask_qna`). The full `crud_qna.py` is disabled on this branch. The response is a single JSON reply with no streaming.



Steps 2–3 and the final step are skipped for `/docs`, `/health`, `/admin/refresh-prompts`, everything under `/ingest` and multipart bodies. A follow-up on an unknown `group_id` returns 404 "Chat group not found".

Request body

`query`, `group_id`, `isNew`, `filters`, `attachment` (S3 key or https URL). The chat type comes from the `filters` keys matched against `goose_chat_types`.

Response body

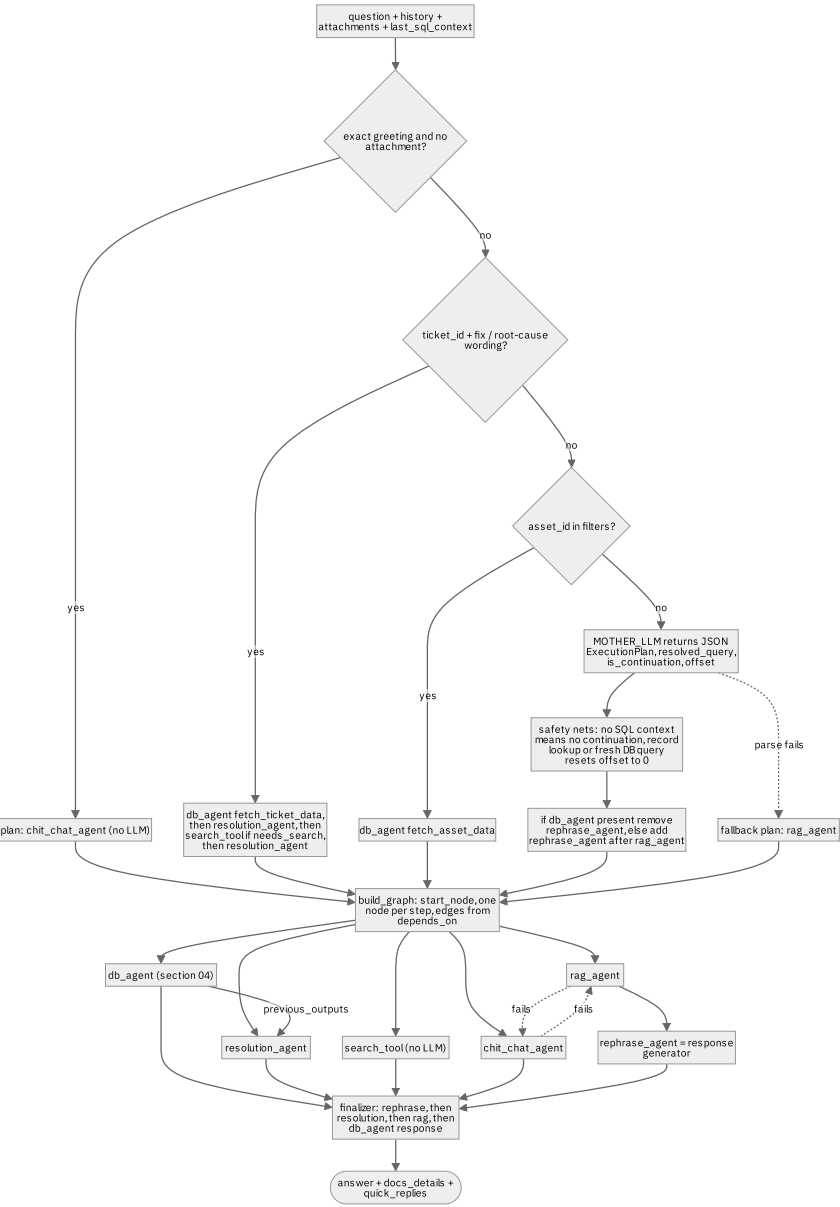
`answer`, `type`, `user_id`, `docs_details`, `group_id`, `group_name`, `quick_replies`, `attachment_path`, `attachment_chunks`, `attachment_error`.

Chat history endpoints

`/qna/chat_groups/list` buckets the org's groups from the last 30 days by date. `/details` pages through the turns. `/rename` and `/delete` (soft, `active` 1 → 0) are PATCH calls.

How a question becomes a plan, and a plan becomes a graph

Data questions about tickets, surveys and assets go to `db_agent` . Document questions go to `rag_agent` . `TaskPlanner.generate_plan()` returns an `ExecutionPlan` : a list of steps, each with `agent` , `action` , `depends_on` and `condition` . `build_graph()` turns that list into a `LangGraph StateGraph` for each request, compiled with the Postgres checkpointer.

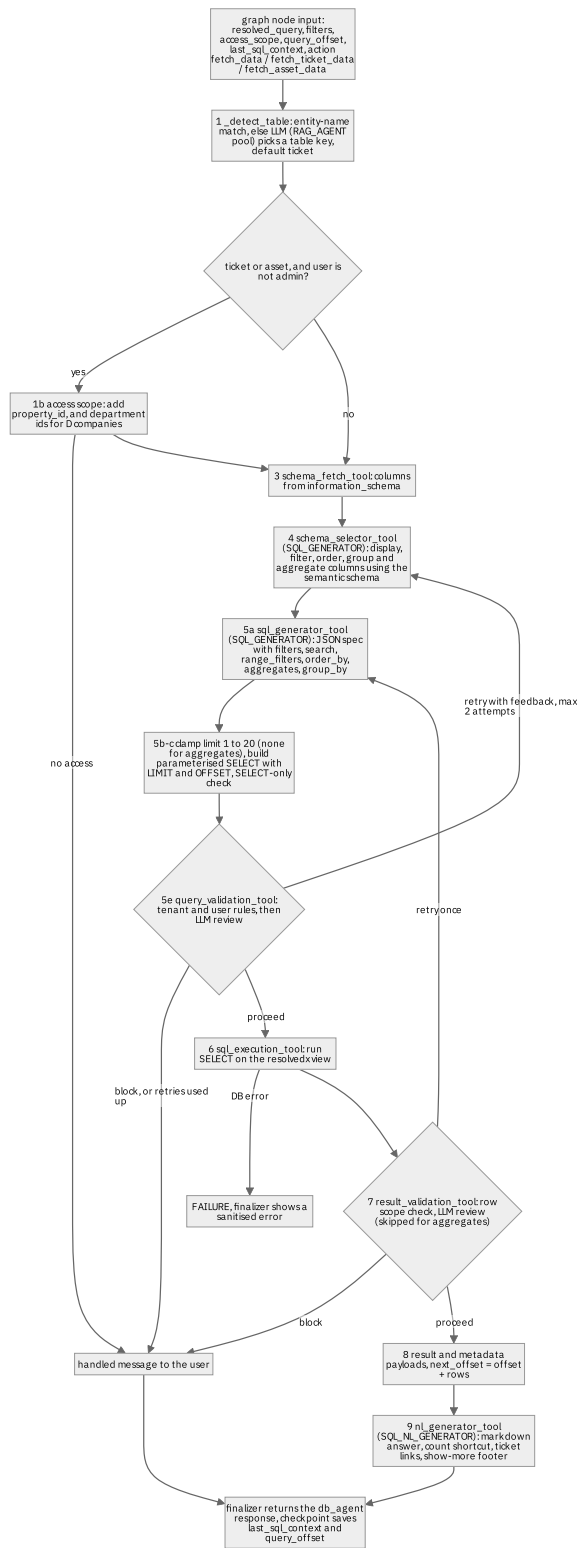


Each step's `condition` is checked before the node runs. `db_agent` writes its own markdown answer, so the planner removes `rephrase_agent` from any plan that contains it. After a successful `db_agent` run the node stores `last_sql_context` and moves `query_offset` forward. That is how "show more" continues the same query. The dotted edges are the built-in fallbacks between `rag_agent` and `chit_chat_agent` .

AGENT	JOB	LLM CONFIG (LLM_AGENTS ROW)	STATE
rag_agent	Runs the 8-tool retrieval pipeline (section 04) and makes one grounded LLM call	RAG_AGENT	live
rephrase_agent	Response generator: formats the final answer and suggested questions	RESPONCE_GENERATOR_AGENT	live
chit_chat_agent	Conversational fallback, greetings	MOTHER_LLM pool, built-in prompt	live
resolution_agent	Ticket troubleshooting	RESOLUTION_AGENT prompt, RESPONCE_GENERATOR pool	live
search_tool	Knowledge-base search with no LLM step	none	live
db_agent	NL-to-SQL on ticket, survey and asset views: detect table, scope access, select columns, generate, validate, execute, write the answer (section 04)	RAG_AGENT pool (table pick), SQL_GENERATOR (columns, query, both validators), SQL_NL_GENERATOR (answer)	live

How a data question becomes a safe, scoped SELECT

The LLM never writes SQL. It fills in a JSON query spec, and `build_query` turns that spec into parameterised SQL against a short list of allowed views. Two validators, each with deterministic rules plus an LLM review, sit before and after execution. Code: `agents/db_agent.py` , `agents/table_semantic_schema.py` , `tools/db/*` .



Step numbers match the code. Step 2 (`DBToolPlanner`) only logs the fixed tool order, and the value-hints step (4a) is switched off. A query-validation retry re-runs the column selector and the generator with the validator's feedback. A result-validation retry re-runs generation once. There is no repair loop for a database error.

TABLE KEY	VIEW (SCHEMA RESOLVEDX)	ACCESS SCOPED	TYPICAL QUESTIONS
ticket	ticket_assignment_view_5	yes	open tickets at a property, tickets assigned to me, count by status, ticket by number
asset	asset_master_view	yes	assets in a building, asset details by id
survey	survey	no	survey responses and scores

Guards before execution

- The LLM only picks a table key. Extra tables can be added through db_agent_tables in Redis goose_ai_extra_config .
- Every column name is checked against the fetched columns. Values are bound as parameters.
- The SQL must start with SELECT. DELETE, UPDATE, INSERT, ALTER, DROP, CREATE, TRUNCATE, GRANT and REVOKE are rejected.
- The query is blocked when it asks for another user's or company's data, "all companies", or a write action.
- The LLM review checks tenant scope, PII, jailbreaks, schema probing and SQL injection.

Access scope

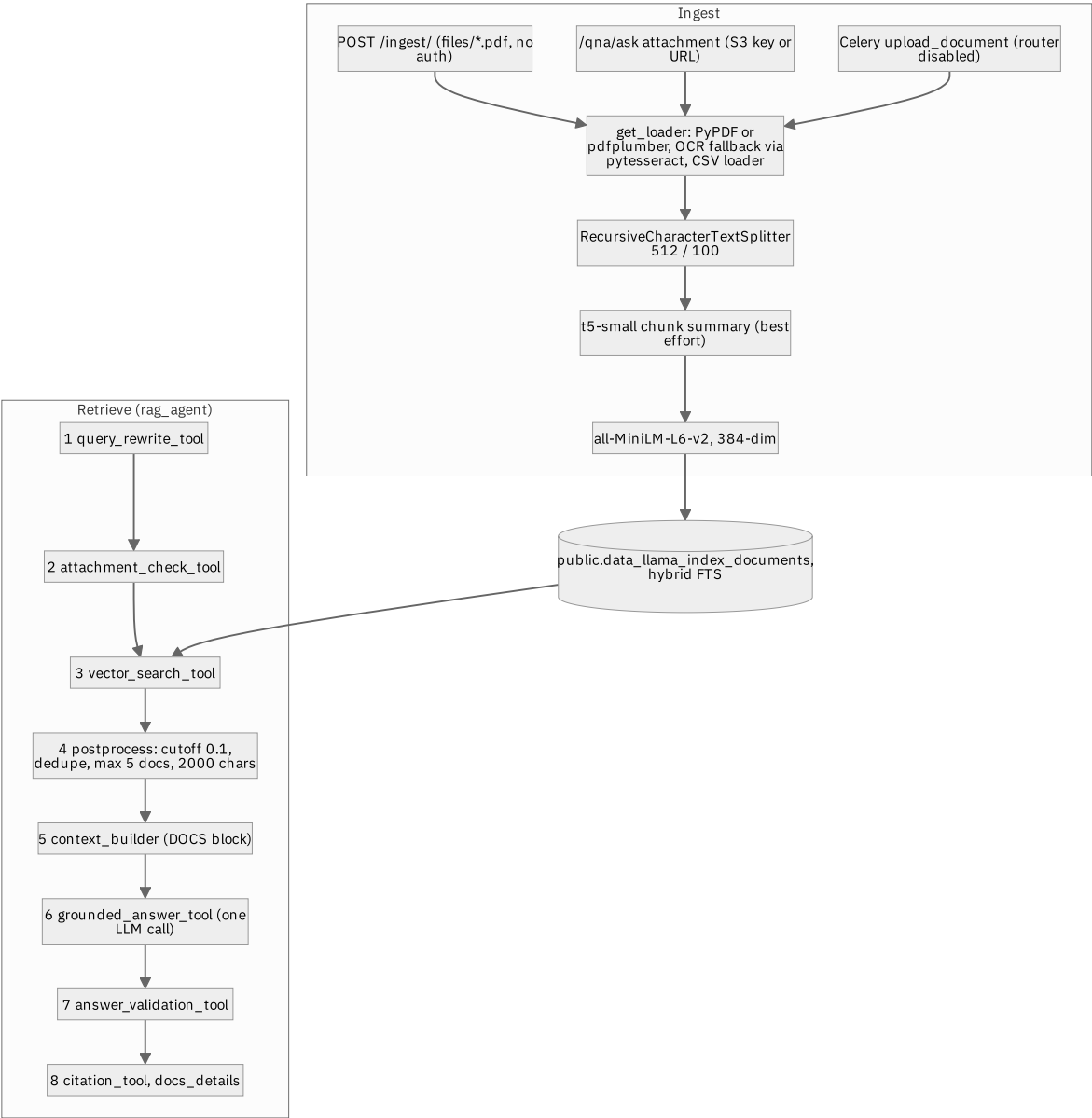
- Resolved once per request by resolve_user_access_scope , from the user's property and department grants.
- For company permission flag P a property grant is enough. For D the user also needs a department grant.
- Admins (user type 1 or 2) skip scoping. Anything unrecognised is denied.
- Rows coming back are checked again: any company or user column must match the caller.

Answer and "show more"

- A single COUNT row is answered from a template ("There are N open tickets.") with no LLM call.
- Otherwise SQL_NL_GENERATOR writes markdown. Ticket markers become encrypted links labelled with display_number .
- When 20 rows come back, a footer says to type **show more**. The planner sees last_sql_context , sets is_continuation , and the same query runs with a larger OFFSET.
- DB answers return type: "text" and an empty docs_details .

Three ways in, one store, two scoped searches

All ingestion runs through `crud_ingest._store_file` . Retrieval is a fixed tool sequence inside `rag_agent` . The access filter (`ACCESS_FILTER_ENABLED = True`) keeps chat attachments private to their own chat.



The `search_documents` and `retrieve_context` actions run the same sequence without steps 6–7. Tunables come from Redis `goose_ai_extra_config`: `k_rag_search` 5, `rag_similarity_cutoff` 0.1, `rag_max_docs` 5, `rag_max_doc_chars` 2000, `rag_llm_retries` 3.

General search (every query)

`group_id IS NULL AND (organization_id = caller_org OR (document_access_type = 2 AND organization_id != ""))`

A super admin has no org, so sees only global documents (type 2). Old chunks with a NULL org are never returned.

Chat search (chat has attachments)

`group_id = this_chat AND organization_id = caller_org`

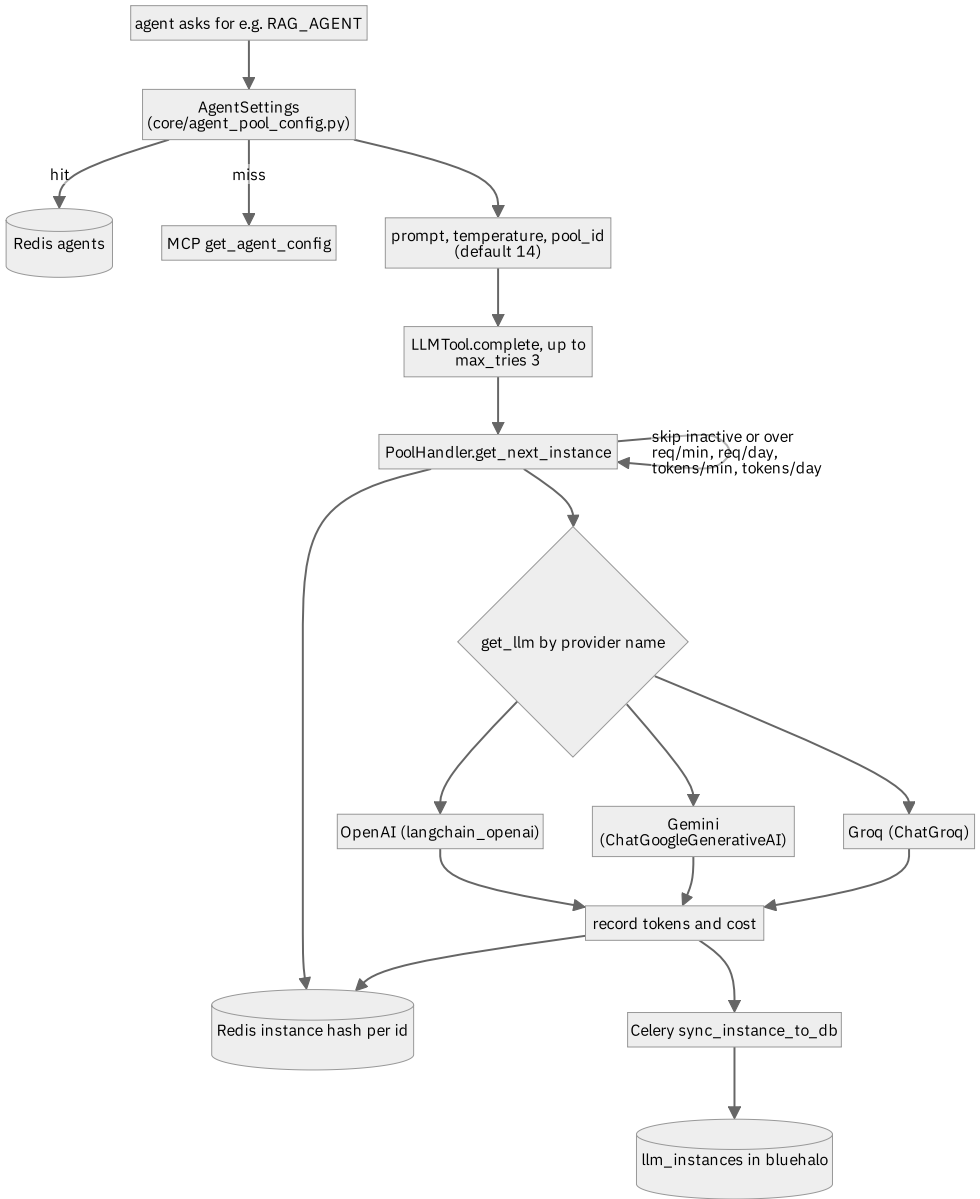
Runs first. `merge_results` puts chat chunks first, dedupes by node id, then sorts by score.

Chunk metadata

`id` , `source` , `document_name` , `upload_date` , `user_ids` , `organization_id` , `document_access_type` (0 user, 1 org, 2 global), `module` , `summary`, and `group_id` for chat attachments. Attachment ids are uuid5 of `group_id:key` .

From an agent name to a provider call

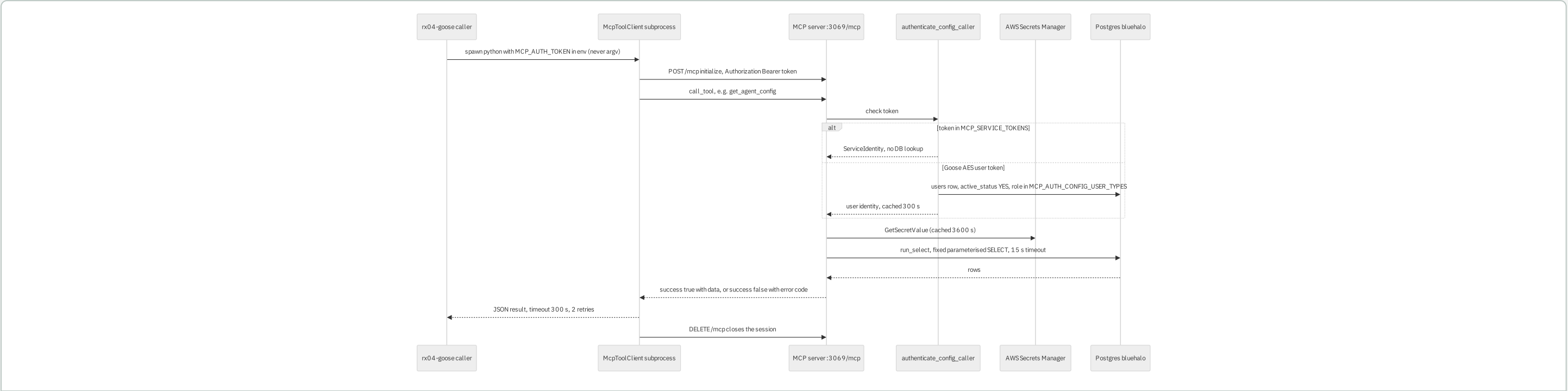
No model name or API key lives in code or config. Each agent row points at an instance pool; the pool handler picks the next instance that is under its limits.



At startup the lifespan hook fills Redis `agents` from MCP `list_agents` and `pool_and_instances` from the pool tables. `POST /admin/refresh-prompts` re-runs the agent cache. Instance API keys are stored AES-encrypted in `llm_instances`. There is no Anthropic or Bedrock provider.

rx04-goose-mcp-server: a locked read-only window onto bluehalo

Built on the official MCP Python SDK (FastMCP). The server exposes 7 tools and no resources or prompts, with no generic SQL tool by design; tests enforce this. It makes no LLM calls, no outbound HTTP calls and does not use Redis.

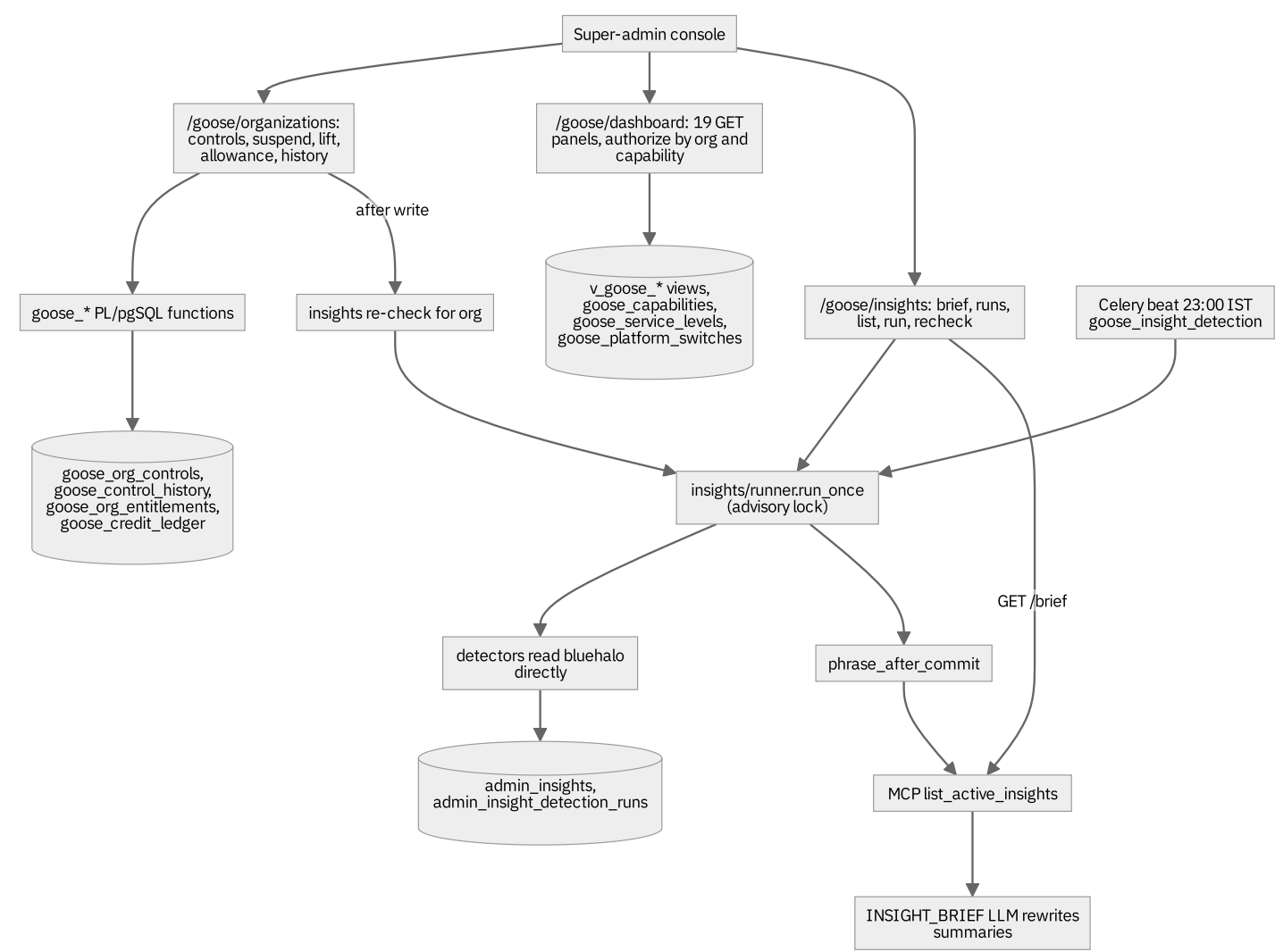


If Postgres rejects the credentials, the server refreshes them from Secrets Manager and retries once. `MCP_AUTH_REQUIRED=false` skips the gate for development only. Errors sent to clients never include raw DB or AWS text.

TOOL	READS	CALLED BY RX04-GOOSE
<code>list_agents()</code>	<code>llm_agents</code> , <code>instance_pools</code>	startup cache warm, <code>agent_services.py</code>
<code>get_agent_config(agent_name)</code>	one agent + its pool's instances (includes <code>api_key</code>)	<code>AgentSettings</code> on Redis miss, prompt renderer
<code>get_pool_instances(pool_id)</code>	pool → instance → model → provider (includes <code>api_key</code>)	available
<code>list_pools_and_instances()</code>	every pool with instances, no keys	available
<code>list_llm_instances(active_only)</code>	<code>llm_instances</code> ⌘ models ⌘ providers (includes <code>api_key</code>)	available
<code>get_extra_config()</code>	<code>goose_ai_extra_config</code> as a key/value map	available
<code>list_active_insights(limit, organization_id, ids)</code>	ACTIVE <code>admin_insights</code> by <code>rank_score</code> , cap 500, no <code>details_json</code>	<code>insights/llm_brief.py</code>

Org controls, dashboard and insights

Every route here requires role 1. Writes go through `goose_*` PL/pgSQL functions rather than ORM updates, and each write triggers an insights re-check for that org.



Insight detectors write `admin_insights` through rx04-goose's own SQLAlchemy session. Only the read whose rows feed the INSIGHT_BRIEF LLM goes through MCP.

What Celery runs, and when

Worker: `celery -A app.workers.celery_workers worker --pool=threads` , plus `beat` .

Broker and result backend are Redis on port 6379 (hard-coded). Schedule times are

Asia/Kolkata.

TASK	TRIGGER	WHAT IT DOES
<code>sync_instance_to_db</code>	<code>.delay()</code> after each LLM call	Copies Redis instance counters (tokens in/out, requests used, active) into <code>llm_instances</code>
<code>upload_document</code> / <code>update_document</code> / <code>delete_document</code>	document router (disabled)	S3 → loader → chunks → embeddings → vector store; update ACL <code>user_ids</code> ; delete vectors
<code>goose_insight_detection</code>	beat, daily 23:00	Runs the insight detectors, then phrases the results with the INSIGHT_BRIEF LLM
<code>reset_request_used_task</code>	beat, daily 15:52	Resets request counters in DB and Redis, sets active from <code>limit_alert_cost</code>
<code>monthly_pattern_detection</code>	beat, 1st of month 12:00	mlxtend association rules on last month's tickets, LLM summary for admins
<code>monthly_anomaly_detection</code>	beat, 9th of month 15:44	Robust z-score (median / MAD) per use case, LLM summary, admin notification
<code>create_today_user_summary</code>	<code>worker_ready</code> signal	LLM summary of each user's chats, written to disk

rx04-goose routers on this branch

This branch is an authentication bring-up build. Routers whose tables are no longer on the direct connection are commented out in `api/api_v1/api.py` until their reads move to MCP.

PREFIX	ROUTES	AUTH	STATE
/qna	POST /ask , POST /chat_groups/list , POST /chat_groups/details , PATCH /chat_groups/rename , PATCH /chat_groups/delete	roles 1, 2, 3	live
/goose/dashboard	19 GET panels: orgs, switches, capabilities, levels, routing, kpis, alerts, and per-org usage, daily, ledger, entitlements, users...	role 1	live
/goose/insights	GET /brief , /runs , list, /{id} ; POST /run , /recheck , /{id}/recheck	role 1	live
/goose/organizations	GET/POST /{org_id}/controls , /controls/options , POST /suspend , /lift , /allowance , GET /history	role 1	live
/llm	CRUD for providers, models, instances, instance pools and pool mappings; read/all/llm_details	role 1	live
/agents	read all, read one, reset prompt, update llm_agents	role 1	live
/ingest	POST / , GET /files	none	live, open
/health , /admin/refresh-prompts	auth + SELECT 1 ; re-cache agents into Redis	health: yes · refresh: none	live
disabled	/auth , /document , /faq , /extra_config , /sentiment , /predict , /text_classification , /ticket_summarization , /ticket_field_suggestions , /api_request_log	—	commented out

What changed in the fork

rx-goose has no reference to either rx04 repository, and nothing calls it. It is useful as a reference for the features rx04-goose has switched off.

CONCERN	RX-GOOSE (LEGACY)	RX04-GOOSE (CURRENT)
Schema	<code>resolvedx</code> : <code>mst_company</code> , <code>efar_user</code> , tickets, FAQs, properties	<code>bluehalo</code> : <code>users</code> , <code>roles</code> , <code>organizations</code> , <code>goose_*</code> ; operational tables to be read through MCP
Auth	Bearer = AES-encrypted user id, hard-coded key	RS256 JWT from the RX04 auth service; AES kept as rollback mode
Transport security	Plain JSON	Encrypted envelope keyed by <code>txnno</code>
Config reads	Direct DB and Redis	Agent config through the MCP server
Prompts	MLflow Prompt Registry → <code>llm_agents</code> → code defaults	<code>llm_agents</code> via MCP; MLflow settings remain but nothing uses them
Chat history	LangGraph checkpointer + per-day Redis keys	<code>goose_chat_group</code> / <code>goose_chat_data</code> + Redis <code>chat_history:*</code> + checkpointer
Agents	<code>db_agent</code> , <code>rag</code> , <code>resolution</code> , <code>search</code> , <code>rephrase</code>	Same set, including <code>db_agent</code> (identical code), plus <code>chit_chat</code> and a greeting shortcut
RAG access	<code>company</code> + <code>user_ids</code> + access type	Org filter plus chat-scoped attachments (<code>group_id</code>)
Extras	FAQ cache, sentiment, BERT ticket classifier, zero-shot, ticket summaries, field suggestions	Same code present but routers disabled; new admin plane (controls, dashboard, insights)

Things to fix or know before relying on the diagram

security

The vector DB connection string is hard-coded in `core/config.py` with a username and password, on host 192.168.5.10. `config.py` also prints the checkpoint connection string, including the DB password, to stdout at import.

security

`/ingest` and `/admin/refresh-prompts` have no authentication.

security

The MCP server uses the main app's privileged DB secret; there is no dedicated read-only DB role yet. Three tools return `api_key`, so the service token is sensitive.

broken path

`graph/data_summary/mcp_nodes.py` and `tools/mcp/tool_selector.py` still call ticket/asset tools the MCP server no longer exposes. Their router is disabled, so nothing hits this today.

to enable

`db_agent` is drawn as working, but on this branch it does not register yet. It imports `Company`, and `utils/access_scope.py` imports `EfarUser`, `CompanyProperty` and other models that are not in `bluehalo`. To match rx-goose: replace those imports with `Organization`; rewrite the access scope; give the schema-fetch and execution tools a connection that can read the `resolvedx` views (or add ticket, asset and survey tools to the MCP server, which allows no raw SQL); map the UUID `organization_id` to the views' `company_id`.

security

In `db_agent`'s query builder, the `having` clause is passed through from the LLM as a raw string and is not parameterised. Super admins have no `organization_id`, so no tenant filter would be added for them.

config

Beat times look like leftover test values: `reset_request_used` at 15:52 (docstring says 00:00) and monthly anomaly detection on the 9th at 15:44.

docs

The MCP README says port 8765; the code and `.env` use 3069. The MCP docs say no client is wired in, but rx04-goose already calls it.

Sources: `rx04-goose/app/app` (main.py, api/api_v1, crud, services, planner, graph, agents, tools/rag, tools/mcp, round_robin, insights, workers), `rx04-goose-mcp-server/mcp_server` and `docs/ARCHITECTURE.md`, `rx-goose/app/app`. No database or server was contacted to build this page.